7e. Functions: Type Inference and Multiple Dispatch

Martin Alfaro
PhD in Economics

INTRODUCTION

In Julia, functions are key for achieving high performance. This is by design, as functions have been engineered from the outset to generate efficient machine code.

However, to fully unlock their potential, we must first understand the underlying process of function calls. Essentially, when a function is called, Julia attempts to identify concrete types for its variables, eventually selecting a corresponding method for computation. At the heart of the process are three interconnected mechanisms: **dispatch**, **compilation**, and **type inference**. This section will provide a detailed explanation of each concept.

VARIABLE SCOPE IN FUNCTIONS

To fully appreciate why functions are essential for performance in Julia, we must first revisit the notion of variable scope. Recall that **local variables** in a function include its arguments and any variable created inside the function body. These variables exist only during the function's execution and are inaccessible from outside. **Global variables**, on the other hand, refer to any variable defined outside a function's scope and remain accessible throughout program execution.

Anticipating the conclusions we'll obtain this section, a key takeaway is that **wrapping code in functions is crucial for high performance in Julia**. Functions ensure that variables are local, which helps to avoid the significant performance penalty introduced by global variables.

Note that the use of global variables isn't limited to code written in the global scope. It also arises when a function references variables that aren't passed as arguments. As a result, the detrimental effect of global variables will appear in all the following cases.

GLOBAL SCOPE

```
x   = 2

y   = 3 * x
```

```
julia> y
6
```

FUNCTION USING A GLOBAL VARIABLE

```
x   = 2

f() = 3 * x
```

```
julia> f()
6
```

Recall that an expression like `x = 2` is shorthand for `x::Any = 2`, reflecting that global variables default to `Any` if they aren't explicitly type-annotated. Also remember that only concrete types can be **instantiated**, meaning that values can only adopt a concrete type. Consequently, `x::Any` shouldn't be understood as `x` having type `Any`, but rather that `x` can take on any concrete type that's a subtype of `Any`. Since `Any` sits at the top of Julia's type hierarchy, this simply means that `x`'s types are unrestricted.

The unrestricted nature of global variables prevents specialization of operations. In our example, this means that a global variable like `x` prevents specialization of `*`. Because of this possibility, Julia must consider multiple possible methods for its computation, one for each possible concrete type of `x`. In practice, this results in Julia generating code with multiple branches, potentially involving type checks, conversions, and object creations. The consequence is degraded performance.

The reason for Julia treating global variables as potentially embodying any type is that, even if a variable holds some value at a specific moment, the user may reassign it at any point in the program. However, even if we had type-annotated `x` with a concrete type like `x::Int64 = 2`, the performance limitations wouldn't completely go away. Certain optimizations can only be implemented when both the scope of variables is clearly delimited and values are known. When this is the case, Julia can gain a comprehensive view of all the operations to be performed, creating opportunities for further optimizations.

Functions in Julia are designed precisely to address these considerations. By enforcing local scope and predictable variable behavior, they allow the compiler to specialize methods and apply powerful optimizations. In the next section, we'll explore the steps Julia takes to achieve this.

FUNCTIONS AND METHODS

A **function** is a name that can be associated with multiple implementations. These implementations are known as methods. A **method** specifies the function body for a particular combination of argument types and number of arguments. The list of methods associated with a function `foo` can be retrieved by executing `methods(foo)`.

To illustrate this, let's define several methods for some function `foo1`. Creating methods requires type-annotating `foo1`'s arguments with the operator `::` during its definition. This strategy makes it possible to set a distinct function body for each unique combination of argument types.

To keep matters simple, let's start considering a scenario where all the methods of `foo1` have the same number of arguments.

METHODS

```
foo1(a,b)                  = a + b
foo1(a::String, b::String) = "This is $a and this is $b"
```

```
julia> methods(foo1)
2 methods for generic function "foo1" from Main
julia> foo1(1,2)
3
julia> foo1("some text", "more text")
"This is some text and this is more text"
```

Since `foo1(a,b)` is equivalent to `foo1(a::Any,b::Any)`, the first method sets the behavior of `foo1` for any combination of input types. This behavior is subsequently overridden by defining the method `foo1(a::String, b::String)`. By doing this, we're providing a specific function body for `a` and `b` with type `String`. The existence of multiple methods explains the differences in outputs obtained: the first method of `foo1` is called with `foo1(1, 2)`, whereas `foo1("some text", "more text")` triggers the second method.

The example also reveals that **methods don't need to perform similar operations**. Although mixing drastically different operations under a single function name isn't recommended, allowing variation in function bodies is relevant for performance. In practice, it's commonly used for adapting computation algorithms to each specific type combination, thus optimizing the overall performance of a function.

Also note that **methods don't need to have the same number of arguments**. For instance, it's possible to define the following methods for a function `foo2`.

METHODS WITH DIFFERENT NUMBERS OF ARGUMENTS

```
foo2(x)       = x
foo2(x, y)    = x + y
foo2(x, y, z) = x + y + z
```

```
julia> methods(foo2)
3 methods for generic function "foo2" from Main
julia> foo2(1)
1
julia> foo2(1, 2)
3
julia> foo2(1, 2, 3)
6
```

Defining methods with different number of arguments is particularly useful for extending a function's behavior. A prime example is given by the function `sum`. So far, we've only used its simplest form `sum(x)`, which adds all the elements of a collection `x`. However, `sum` also supports additional methods. One of them is `sum(<function>, x)`, where the elements of `x` are transformed via `<function>` before being summed.

METHODS FOR 'SUM'

```
x = [2, 3, 4]

y = sum(x)          # 2 + 3 + 4
z = sum(log, x)     # log(2) + log(3) + log(4)
```

FUNCTION CALLS

Building upon our understanding of function definition and methods, let's now analyze the process triggered when a function is called. All our explanations will be based on the following function `foo`:

EXAMPLE

```
foo(a, b) = 2 + a * b
```

```
julia> foo(1, 2)
4
julia> foo(3, 2)
8
julia> foo(3.0, 2)
8.0
```

In Julia, defining a function like `foo(a, b)` is shorthand for creating a **method** with the signature `foo(a::Any, b::Any)`. Thus, the function body `foo(a,b)` holds for all possible type combinations of `a` and `b`.

When `foo(1, 2)` is called, Julia evaluates the expression `2 + a * b` by following a series of steps.

The process begins with what's known as **multiple dispatch**, where Julia selects which method of a function to execute. Importantly, this decision is based solely on the types of the arguments, not their values. Specifically, Julia begins by identifying the concrete types of the function arguments. In our example where `a = 1` and `b = 2`, both are identified as `Int64`. The information on types is then used to select a *method*, which defines the function body and hence the operations to be performed. This process involves searching through the available methods of `foo` to find the most specific one for the concrete types of `a` and `b`. In our example, `foo` has only one method `foo(a,b) = 2 + a * b`, which is defined for all argument types, including `a::Int64` and `b::Int64`. Therefore, the corresponding function body is `2 + a * b`.

The specific version of this method for the signature `foo(a::Int64, b::Int64)` is known as a **method instance**. If the code for the method instance `foo(a::Int64, b::Int64)` already exists, Julia will directly employ it to compute `foo(1,2)`. Otherwise, the compiler generates optimized code for that method instance, stores it in memory for future use, and Julia executes it.

The following diagram depicts the process unfolded when `foo(1,2)` is executed.

MULTIPLE DISPATCH

The process outlined has implications for how the language works. When a function is called with a particular combination of argument types for the first time, Julia incurs an additional cost because it must generate specialized code for those types. This initial delay is often referred to as **Time To First Plot**, a phrase that highlights how the compilation overhead becomes noticeable in interactive workflows such as plotting. Once the code has been compiled, however, Julia stores the resulting method instance, so that subsequent calls with the same argument types can reuse it. This eliminates the compilation step and leads to much faster execution.

The example with `foo` illustrates this process clearly. After evaluating `foo(1, 2)`, Julia has already compiled a method instance for the signature `foo(a::Int64, b::Int64)`. This is why the subsequent call `foo(3, 2)` can be executed immediately by invoking the cached method instance, without any need for recompilation. Instead, the execution of `foo(3.0, 2)` introduces a new combination of argument types, where `a::Float64` and `b::Int64`. Because no compiled method instance yet exists for this signature, Julia must generate one before executing the function.

TYPE INFERENCE

Most considerations for achieving high performance are related to the compilation process. In particular, Julia employs **Just-In-Time Compilation (JIT)**, a term reflecting that compilation happens on the fly during the function call.

A key mechanism in this process is **type inference**, whereby the compiler attempts to identify concrete types for *all* variables and expressions. If the compiler succeeds in identifying concrete types, it can specialize instructions for each operation and yield fast code.

This is the essence behind **type stability**, which we'll cover extensively in the next chapter. For instance, type inference in our example involves determining concrete types for `2`, `a = 1`, and `b = 2`. Since all values have type `Int64`, the compiler can specialize the computation of `2 + a * b` for variables with type `Int64`.

On the contrary, if the compiler is unable to identify concrete types for some expressions, it must create generic code to accommodate multiple combinations of types. This forces Julia to perform type checks and conversions during runtime, significantly degrading performance.

REMARKS ON TYPE INFERENCE

Below, we provide various remarks about type inference that are worth keeping in mind for next sections.

FUNCTIONS DO NOT GUARANTEE IDENTIFICATION OF CONCRETE TYPES

Merely wrapping code in a function doesn't guarantee that the compiler will identify concrete types for all operations. The following example illustrates this.

TYPE-UNSTABLE FUNCTION

```
x       = [1, 2, "hello"]     # Vector{Any}

foo3(x) = x[1] + x[2]         # type unstable
```

```
julia> foo3(x)
3
```

The issue in the example is that the compiler assigns the type `Any` to both `x[1]` and `x[2]`, as they come from an object with type `Vector{Any}`. As a consequence, the compiler can't specialize the computation of the operation `+`. The example also highlights that compilation is exclusively based on types, not values. Thus, the generated code ignores the actual values `x[1] = 1` and `x[2] = 2`, which would otherwise reveal the type `Int64`.

GLOBAL VARIABLES INHERIT THEIR GLOBAL TYPE

Type inference is restricted to local variables, with any global variable having its type inherited from the global scope. For instance, consider the following example.

UNANNOTATED GLOBAL VARIABLE

```
a         = 2
b         = 1

foo4(a)   = a * b
```

```
julia> foo4(a)
2
```

TYPE-ANNOTATED GLOBAL VARIABLE

```
c         = 2
d::Number = 1

foo4(c)   = c * d
```

```
julia> foo4(c)
2
```

In the examples, `b` and `d` are a global variable. Consequently, the compiler infers `b`'s type as `Any` and `d` as `Number`.

TYPE-ANNOTATING FUNCTION ARGUMENTS DOES NOT IMPROVE PERFORMANCE

Identifying concrete types is crucial for achieving high performance. At first glance, this might suggest that explicitly annotating function arguments is necessary for performance, or at least beneficial. However, thanks to type inference, such annotations are redundant. In fact, adding them can be

counterproductive, as they unnecessarily restrict the types accepted by a function, thereby limiting its flexibility and potential applications.

To better appreciate this loss of flexibility, compare the following scripts.

UNANNOTATED FUNCTION

```
foo5(a, b) = a * b
```

```
julia> foo5(0.5, 2.0)
1.0
julia> foo5(1, 2)
2
```

TYPE-ANNOTATED FUNCTION

```
foo6(a::Float64, b::Float64) = a * b
```

```
julia> foo6(0.5, 2.0)
1.0
julia> foo6(1, 2)
ERROR: MethodError: no method matching foo6(::Int64, ::Int64)
```

The function on the first tab only accepts arguments of type `Float64`, implying that even integers are disallowed. By contrast, the function on the second tab mirrors the behavior with `Float64` inputs, but additionally allows for other types. This flexibility stems from the implicit type annotation `Any` on the function arguments.

> **Packages Commonly Type-Annotate Function Arguments**
>
> When inspecting code of packages, you may notice that function arguments are often type-annotated. The reason for this isn't related to performance, but rather to ensure the function's intended usage, safeguarding against inadvertent type mismatches.
>
> For instance, suppose a function that computes the revenue of a theater via `nr_tickets * price`. Importantly, the operator `*` in Julia not only implements product of numbers, but also concatenates words when applied to expressions with type `String`. Without type-annotations, the function could potentially be misused, as exemplified in the first tab below. The second tab precludes this possibility by asserting types.
>
> **UNANNOTATED FUNCTION**
>
> ```
> revenue1(nr_tickets, price) = nr_tickets * price
> ```
>
> ```
> julia> revenue1(3, 2)
> 6
> julia> revenue1("this is ", "allowed")
> "this is allowed"
> ```

TYPE-ANNOTATED FUNCTION

```
revenue2(nr_tickets::Int64, price::Number) = nr_tickets * price
```

```
julia> revenue2(3, 2)
6
julia> revenue2("this is ", "not allowed")
ERROR: MethodError: no method matching revenue2(::String,
::String)
```